

UNIWERSYTET ŚLĄSKI W KATOWICACH  
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH

**Kamil Dziura**

345550

**Opracowanie i realizacja aplikacji wspomagającej analizę ruchu  
drogowego poprzez pomiar natężenia ruchu**

PRACA DYPLOMOWA  
INŻYNIERSKA

dr inż. Tomasz Wesołowski

Katowice 2025

## SPIS TREŚCI

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Wstęp .....                                              | 3  |
| 1.1 Cel.....                                                | 3  |
| 1.2 Zastosowanie .....                                      | 4  |
| 1.3 Przegląd istniejących systemów .....                    | 4  |
| 1.3.1 Systemy bazujące na kamerach.....                     | 4  |
| 1.3.2 Systemy bazujące na detektorach .....                 | 6  |
| 1.3.3 Aplikacje mobilne i komputerowe .....                 | 8  |
| 2. Analiza zadania i projekt systemu .....                  | 9  |
| 2.1 Dziedzina pracy .....                                   | 9  |
| 2.2 Analiza zadania.....                                    | 10 |
| 2.3 Wymagania .....                                         | 11 |
| 2.3.1 Wymagania funkcjonalne .....                          | 11 |
| 2.3.2 Wymagania нефункционалне.....                         | 13 |
| 2.4 Wykorzystane technologie i biblioteki .....             | 14 |
| 2.4.1 Język Python .....                                    | 14 |
| 2.4.2 Biblioteka OpenCV .....                               | 15 |
| 2.4.3 Model YOLO .....                                      | 16 |
| 2.4.4 Biblioteka Torch .....                                | 16 |
| 2.4.5 Biblioteka Tkinter .....                              | 17 |
| 2.4.6 Biblioteka Pandas.....                                | 17 |
| 3. Struktura i funkcjonalność aplikacji .....               | 19 |
| 3.1 Architektura aplikacji .....                            | 19 |
| 3.2 Interfejs użytkownika (gui.py) .....                    | 20 |
| 3.3 Moduł analizy i detekcji obiektów (analysis.py).....    | 20 |
| 3.4 Moduł śledzenia obiektów (tracker.py) .....             | 21 |
| 3.5 Inne moduły systemu (model_handler.py i utils.py) ..... | 23 |
| 4. Działanie systemu i analiza rozwiązań .....              | 24 |
| 4.1 Graficzny interfejs użytkownika .....                   | 24 |
| 4.2 Algorytm detekcji obiektów .....                        | 26 |
| 4.3 Działanie mechanizmu opartego na linii zliczania .....  | 31 |
| 4.4 Tryb „Record” – zapis danych.....                       | 33 |
| 5. Podsumowanie .....                                       | 36 |
| 5.1. Wnioski .....                                          | 36 |
| 5.2 Plan rozwoju systemu oraz możliwe zastosowania.....     | 37 |
| 6. Bibliografia.....                                        | 38 |

## 1. WSTĘP

W ciągu kilku ostatnich lat zaobserwować możemy dynamiczny wzrost natężenia ruchu drogowego w Polsce. Według badań przeprowadzonych przez Generalną Dyрекcję Dróg Krajowych i Autostrad w latach 2020/2021 średnia dobową liczbą pojazdów korzystających z dróg krajowych powiększyła się o około 21% w porównaniu do badań przeprowadzonych w 2015 roku. Był to także pierwszy tak znaczący wzrost od czasu zmian administracyjnych w 2000 roku – w okresie do 2015 roku średnie natężenie ruchu drogowego pozostawało na relatywnie stabilnym poziomie [1].

Przyczyn takiego stanu rzeczy możemy dopatrywać się w wielu czynnikach. Najważniejszymi z nich jest rozwój gospodarczy i urbanizacja. Dynamiczny rozwój polskiej gospodarki w ostatnich dziesięcioleciach doprowadził do zwiększenia zamożności społeczeństwa. Im większa zamożność, tym więcej osób może sobie pozwolić na zakup samochodu oraz korzystanie z niego kosztem środków transportu zbiorowego. Według danych Głównego Urzędu Statystycznego w ostatnich latach obserwujemy systematyczny wzrost zarejestrowanych pojazdów – pod koniec 2023 roku łączna liczba zarejestrowanych pojazdów wyniosła ponad 27 milionów [2]. Duża urbanizacja również nie pozostaje bez wpływu na natężenie ruchu drogowego. Największe polskie miasta ciągle się rozbudowują, przez co odległość między różnymi częściami miasta staje się coraz większa. Nie wszędzie dostępny jest komfortowy dojazd komunikacją miejską, dlatego też mieszkańcy zmuszeni są do dojazdu samochodami.

Z tego też powodu ważne jest inwestowanie w rozwój systemów zaprojektowanych do zarządzania ruchem drogowym. Wprowadzenie takich systemów może usprawnić ruch drogowy w największych miastach, znacznie redukując korki. Jednym z takich projektów jest Krajowy System Zarządzania Ruchem (KSZR), tworzony przez Generalną Dyрекcję Dróg Krajowych i Autostrad. Jednakże sam projekt nie jest na ten moment zakończony.

### 1.1 CEL

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji służącej do analizy ruchu drogowego poprzez zliczanie pojazdów występujących w strumieniu wideo. Aplikacja

została stworzona w języku Python i wykorzystuje zaawansowane technologie rozpoznawania i śledzenia pojazdów zarówno w czasie rzeczywistym, jak i poprzez analizę dostępnego materiału w postaci nagrania wideo. Stworzone narzędzie dodatkowo pozwala określić kierunek poruszania się pojazdów, co jest kluczowe w kontekście rosnącego zapotrzebowania na inteligentne systemy zarządzania ruchem, które pomagają minimalizować korki oraz ułatwiają planowanie infrastruktury drogowej.

## **1.2 ZASTOSOWANIE**

System realizowany w ramach tej pracy został stworzony m. in. z myślą o implementacji go w istniejącym już systemie zarządzania ruchem drogowym. Został on jednak stworzony w taki sposób, że może funkcjonować również samodzielnie. Nie traci on wtedy żadnej funkcjonalności, jest przystosowany do działania na każdym urządzeniu z zainstalowanym środowiskiem Python.

## **1.3 PRZEGLĄD ISTNIEJĄCYCH SYSTEMÓW**

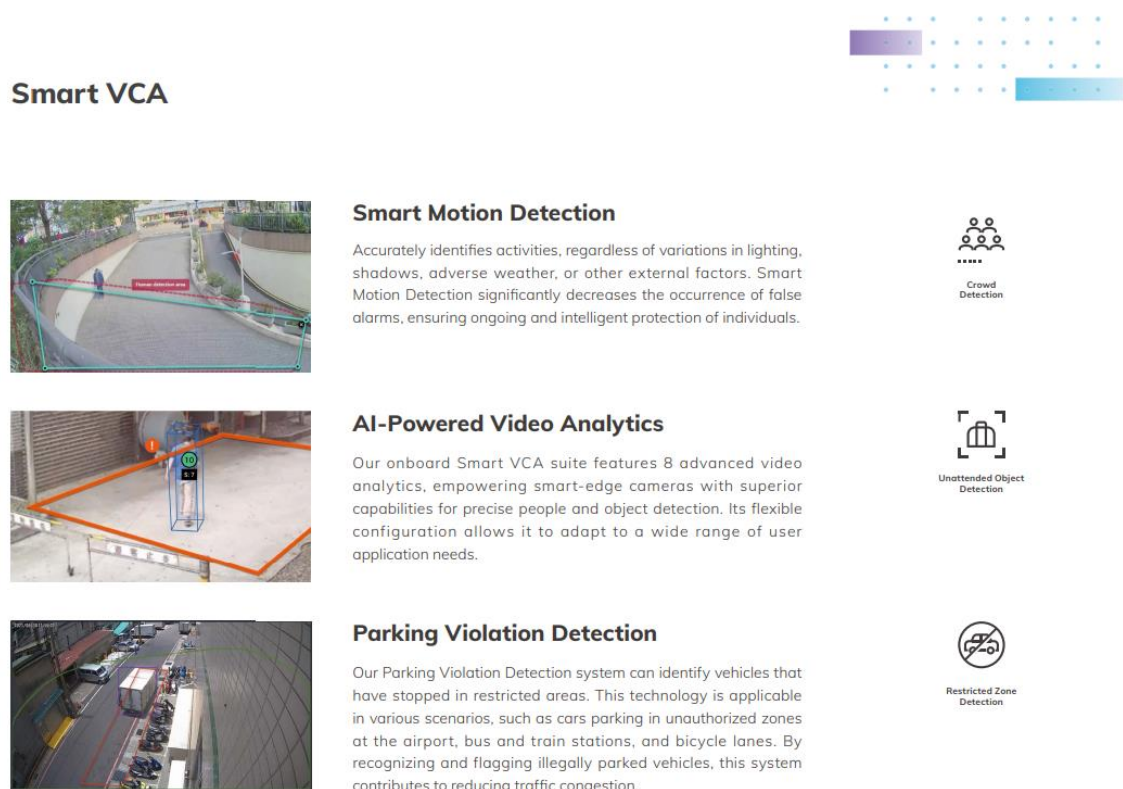
Na rynku istnieje szeroki przekrój aplikacji i systemów zaprojektowanych w podobnym celu co zaprojektowany przeze mnie program. Do ich zadań należy nie tylko liczenie pojazdów, lecz także między innymi identyfikacja problemów takich jak korki bądź też wypadki drogowe oraz badanie przepływu ruchu drogowego. Podczas szukania gotowych rozwiązań podzieliłem te systemy na trzy rodzaje:

1. System kamer wraz z kontrolerem lub wbudowanym komputerem
2. System detektorów lub sensorów
3. Aplikacje mobilne lub komputerowe

### **1.3.1 SYSTEMY BAZUJĄCE NA KAMERACH**

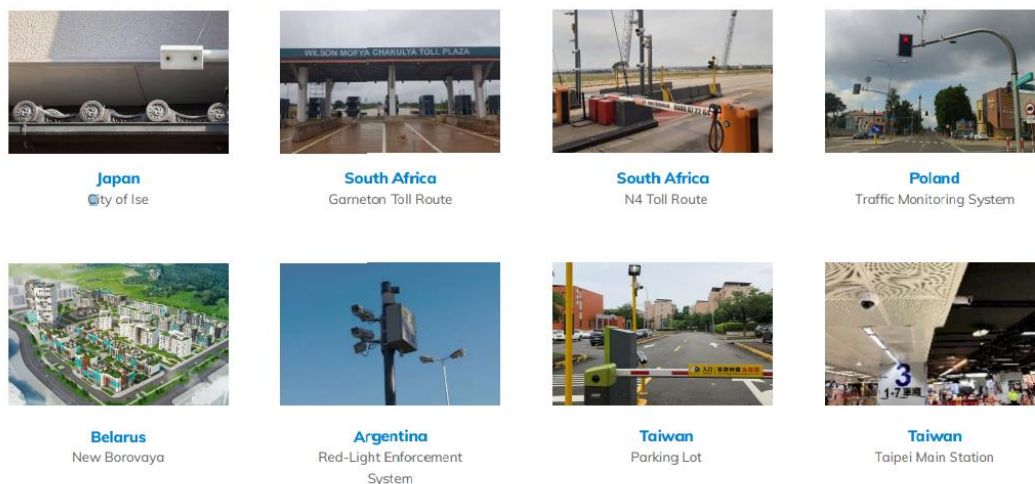
Pierwszy z nich bazuje na analizie wideo poprzez specjalistyczne kamery wraz z odpowiednim oprogramowaniem. Ten rodzaj systemów jest najbliższy zaproponowanemu przeze mnie

rozwiązaniu, aczkolwiek jest dużo bardziej zaawansowany. Kamery rejestrują nagrywany obraz, który następnie jest przetwarzany w czasie rzeczywistym przez komputer zintegrowany z kamerą. Do najpopularniejszych rozwiązań w tym segmencie należą produkty firm Flir [3] oraz Vivotek. Pierwsza marka jest producentem prostszych urządzeń, których głównym zadaniem jest analiza ruchu drogowego, natomiast firma Vivotek [4] jest producentem całych ekosystemów związanych z między innymi wykrywaniem ruchu, analizą wideo z wykorzystaniem algorytmów sztucznej inteligencji oraz wykrywaniem wykroczeń związanych z parkowaniem (Rys. 1). Firmie Vivotek udało się zaimplementować swoje rozwiązania w wielu krajach, do których należy m.in. Polska (Rys. 2). Pomimo wyższego zaawansowania wyżej opisanych systemów, wspólnym mianownikiem między tymi rozwiązaniami a moim systemem jest użycie uczenia maszynowego w celu stworzenia klasyfikatora zdolnego do rozpoznawania pojazdów z klatek wideo.



Rysunek 1 - Przykłady zastosowań systemu oferowanego przez firmę Vivotek [5]

## Success Cases



Rysunek 2 - Przykłady zastosowań technologii formy Vivotek na świecie [5]

### 1.3.2 SYSTEMY BAZUJĄCE NA DETEKTORACH

Drugi rodzaj aplikacji bazuje na fizycznych detektorach i sensorach. Jedną z firm, która specjalizuje się w tworzeniu takich produktów jest francuska firma Eco-Counter. Rozwiązania stworzone przez tę firmę korzystają między innymi z czujników podczerwieni, które pozwalają na detekcję oraz zliczanie pieszych i rowerzystów (system PYRO [6]), oraz z pętli indukcyjnych w celu detekcji pojazdów (system MULTI [7]). Rozwiązanie MULTI Cars/Buses jest szczególnie zaawansowanym wariantem tego systemu, zaprojektowanym z myślą o monitorowaniu ruchu samochodów osobowych oraz autobusów. Dzięki zastosowaniu pętli indukcyjnych (Rys. 3) system jest w stanie precyzyjnie zliczać pojazdy oraz klasyfikować je na konkretne rodzaje. Rozwiązanie te znajduje zastosowanie m.in. na drogach o mniejszym natężeniu ruchu, w celu podziału przejeżdżających pojazdów na pojazdy osobowe i autobusy, oraz przy wjazdach na parkingi, pozwalając na precyzyjne oszacowanie liczby odwiedzających daną placówkę. Opisane rozwiązanie jest szeroko stosowane we Francji, przykładowo w parkach narodowych.

Niestety ten system pomimo swoich zalet nie sprawdziłby się przy analizie natężenia ruchu na obszarach, przez które przejeżdża więcej pojazdów, takich jak autostrady, drogi ekspresowe czy węzły komunikacyjne o dużym natężeniu ruchu. Wynika to z ograniczeń technologii pętli indukcyjnych, które w przypadku bardzo dużej liczby pojazdów mogą mieć trudności z precyzyjnym liczeniem i klasyfikacją wszystkich jednostek w czasie rzeczywistym. Dodatkowym wyzwaniem w takich miejscach jest potrzeba jednoczesnego monitorowania kilku pasów ruchu, co wymagałoby instalacji wielu pętli indukcyjnych i skomplikowanej infrastruktury, znacznie zwiększając koszty implementacji i utrzymania systemu.



*Rysunek 3 - Pętla indukcyjna umieszczona w jezdni*



### 1.3.3 APLIKACJE MOBILNE I KOMPUTEROWE

Trzecim i ostatnim rodzajem aplikacji są proste aplikacje, które zliczają przedmioty ze zdjęcia na podstawie danego wzorca. Jedną z aplikacji, która zwróciła moją uwagę to CountThings autorstwa Dynamic Ventures INC [8]. Jest to aplikacja mobilna, która potrafi przeliczyć powtarzające się przedmioty na zdjęciu. Producenci również zapewniają współpracę aplikacji nie tylko ze zdjęciami, ale również umiejętność przetwarzania filmów wideo. Na stronie internetowej opisującej tę aplikację jest zamieszczony film, który przedstawia działanie aplikacji w jednakowej dziedzinie co mój system, czyli przetwarzanie filmu wideo w celu analizy natężenia ruchu drogowego (Rys. 4). Jednakże przez wysoką cenę zakupu licencji na korzystanie z tej aplikacji nie przetestowałem jej działania, przez co nie jestem w stanie ocenić jej skuteczności w porównaniu z moim systemem. Mimo, iż zarówno ta aplikacja, jak i systemy opisane w punkcie 1.3.1 korzystają z kamery wideo w celu analizy, to różnią się pozostałymi elementami ekosystemu. W przypadku wcześniej opisanych systemów producent oferuje je wyłącznie z jednostką obliczeniową, w przeciwieństwie do tej aplikacji – tutaj obliczenia dokonywane są korzystając z telefonu użytkownika. Z tego też powodu postanowiłem rozgraniczyć te rozwiązania na dwie osobne kategorie.



*Rysunek 4 - Przykład zastosowania aplikacji CountThings do pomiaru ruchu drogowego*



## **2. ANALIZA ZADANIA I PROJEKT SYSTEMU**

W kolejnych podrozdziałach zostanie objaśniona dziedzina pracy oraz dokonany zostanie przegląd istniejących rozwiązań. Zostaną również opisane wybrane technologie użyte podczas implementacji aplikacji wraz z argumentacją dokonanych wyborów. Przeprowadzona zostanie również analiza zadania wraz z opisem możliwych wyzwań i trudności.

### **2.1 DZIEDZINA PRACY**

Niniejszy system związany jest z analizą ruchu drogowego, co jest bardzo istotnym elementem systemów monitorowania transportu oraz zarządzania infrastrukturą miejską. Istniejące już rozwiązania bazują albo na różnych detektorach i sensorach, albo na zaawansowanych kamerach monitorujących wraz ze specjalistycznym oprogramowaniem. Dlatego na początku projektowania aplikacji postanowiłem zbadać, czy jest możliwa zamiana wyżej wspomnianych dedykowanych urządzeń i systemów na względnie proste rozwiązanie wykorzystujące kamerę internetową lub kamerę telefonu, wspieraną poprzez odpowiednie, dedykowane oprogramowanie (przykładowo skrypt w języku Python). Ważne też było przeanalizowanie skuteczności takiego rozwiązania – w końcu system zliczający co drugi pojazd, bądź możliwy do uruchomienia tylko na ekstremalnie drogim komputerze, byłby niepraktyczny i niepotrzebny.

Dużym ułatwieniem podczas projektowania programu był szybki rozwój technologii związanych z przetwarzaniem i analizą obrazu w ostatnich latach, takich jak między innymi szeroko pojęta sztuczna inteligencja oraz uczenie maszynowe. Mimo, iż te terminy były w nauce znane od lat 50 XX wieku (termin „sztuczna inteligencja”, SI został zdefiniowany w 1955 roku przez amerykańskiego naukowca, Johna McCarthy’ego [9]) to dopiero od niedawna możemy zaobserwować rosnącą popularność SI w różnych gałęziach gospodarki. W zaprojektowanej aplikacji również jest ona wykorzystywana, w postaci tak zwanego klasyfikatora. Ogólnie mówiąc, klasyfikator to model, który uczy się rozpoznawać wzorce i generalizować te wzorce do nowych, nieznanych obserwacji. Najczęściej stosuje się go do przekształcania danych wejściowych na dane wyjściowe, które reprezentują przynależność do określonych klas [10]. Dzięki wcześniejszym przetworzeniu przez dany model dużej ilości

danych jest on w stanie z bardzo dużą dokładnością rozpoznawać pojazdy. Ponadto, bardzo skutecznie klasyfikuje je na różne kategorie.

## 2.2 ANALIZA ZADANIA

Przed przystąpieniem do tworzenia systemu konieczne było przeprowadzenie dokładnej analizy zadania, aby wstępnie zaprojektować rozwiązania techniczne oraz zidentyfikować najważniejsze wyzwania oraz problemy.

Po uruchomieniu aplikacji konieczne będzie wywołanie GUI oraz zadeklarowanie wszystkich potrzebnych zmiennych, które będą przechowywać dane dotyczące analizy. Niektóre zmienne będą przyjmować wartość podaną przez użytkownika (takie jak na przykład wybór trybu działania), a niektóre będą miały stałą wartość, niedostępną do konfiguracji (tak jak na przykład liczniki, które po uruchomieniu aplikacji będą przyjmować wartość 0). Po zatwierdzeniu opcji konfiguracyjnych, aplikacja musi rozpocząć proces analizy.

Podczas projektowania systemu wiele uwagi poświęciłem sposobowi przetwarzania wideo. Po analizie dostępnych rozwiązań zdecydowałem się na analizę sygnału wideo klatka po klatce. Takie rozwiązanie jest dostępne i możliwe do zrealizowania w przystępny sposób dzięki wykorzystaniu biblioteki OpenCV. Po odczytaniu klatki można traktować ją jako obraz i zastosować metody analizy obrazów. W celu realizacji stawianych pracy wymagań konieczna jest detekcja obiektów, aby wyodrębnić pojazdy spośród innych obiektów. Do tego celu postanowiłem wykorzystać klasyfikator, w tym przypadku model YOLO. Pozwoli on uzyskać listę klas obiektów z danej klatki wideo, na której następnie będzie można przeprowadzić kolejne działania.

W związku z wybranym sposobem przetwarzania wideo, pojawiło się kilka głównych kwestii, które należało rozważyć. Pierwszą z nich jest sposób na zliczanie pojazdów podczas całego nagrania. Jako, że film jest przetwarzany jako zbiór osobnych klatek, to system bez dodatkowych usprawnień zwróci użytkownikowi tylko i wyłącznie liczbę rozpoznanych pojazdów na konkretnej klatce wideo. Nie będzie możliwe ustalenie całkowitej liczby pojazdów, które przejeżdżały na nagraniu. W tym celu konieczne będzie stworzenie modułu zajmującego się śledzeniem pojazdów oraz zapamiętywaniem ich stanu. Pozwoli to na połączenie kolejnych klatek wideo – następnie, po detekcji system najpierw będzie sprawdzać,

czy dany pojazd został już wykryty wcześniej, czy nie. Należy również rozważyć możliwość wystąpienia wielokrotnych wykryć tego samego pojazdu. Ten problem można wyeliminować na dwa sposoby. Pierwszym z nich jest implementacja dokładniejszego klasyfikatora YOLO. W przypadku tej aplikacji będzie to niemożliwe, ze względu na postawione wymagania dotyczące szybkości przetwarzania. Drugim sposobem jest ograniczenie obszaru, na którym detekcja będzie skutkować aktualizacją statystyk. Powinno to pozwolić na ograniczenie wpływu błędnych detekcji na zgromadzone statystyki, gdyż pojazdy oraz inne obiekty błędnie zaklasyfikowane jako pojazdy nie będą zwiększać stanu liczników. Ostatnim wyzwaniem, które należy rozważyć jest optymalizacja czasu przetwarzania. W przypadku minuty filmu nagranych w 30 klatkach na sekundę, jest to 1800 odrębnych klatek, które system musi po kolei odczytać, dokonać analizy oraz zwrócić wyniki. Cały proces może pochłaniać dużo zasobów komputera, co wydłuży czas analizy. Można jednak ten proces znacząco przyspieszyć, wprowadzając mechanizm pomijania każdej n-tej klatki wideo. Wprowadzenie tej metody jest możliwe dzięki silnej korelacji w czasie konkretnych klatek. Oznacza to, że różnice pomiędzy kolejnymi klatkami wideo są niewielkie, a zmiany położenia pojazdów zachodzą w sposób względnie przewidywalny. Dzięki pominięciu klatek system będzie musiał przetworzyć tylko część klatek, co pozwoli na przyspieszenie całego procesu.

## **2.3 WYMAGANIA**

Poniżej znajdują się wszystkie wymagania funkcjonalne oraz нефункционалне, które zostały określone na etapie definicji celów, analizy zadania i projektowania systemu.

### **2.3.1 WYMAGANIA FUNKCJONALNE**

W celu uporządkowania wymagań funkcjonalnych zostaną one podzielone na kilka podpunktów, związanych z różnymi elementami działania systemu.

a) Konfiguracja modułu analizy:

- Wybór źródła strumienia wideo – użytkownik systemu powinien mieć możliwość wyboru źródła strumienia wideo. Zostało przewidziane zarówno obsługiwanie plików wideo w najpopularniejszych formatach (takich jak na przykład MP4), jak i strumienia wideo na żywo z kamery.
- Ustawienie linii progowej (zliczania) – aplikacja powinna umożliwiać wybór i orientację obszaru, w którym następuje zliczanie pojazdów. Linia ta może być zorientowana pionowo bądź poziomo i musi być umieszczona w określonym miejscu.
- Wybór typów pojazdów – użytkownik powinien mieć wybór, które typy pojazdów mają być zliczane. Opcje, które zostaną zaimplementowane to samochody, motocykle oraz ciężarówki.
- Definicja trybu pracy – aplikacja powinna umożliwiać pracę w dwóch trybach. Przewidziany został tryb „Display”, który będzie wyświetlał wyniki analizy oraz strumień wideo na ekranie oraz tryb „Record”, który będzie umożliwiał zapis wyników do pliku z pominięciem wyświetlania ich.
- Wybór długości interwału pomiaru – użytkownik powinien mieć opcję wyboru interwału resetowania licznika. Powinna być również zadeklarowana domyślna wartość, która będzie wykorzystywana w przypadku braku wyboru użytkownika.

b) Analiza wideo

- Detekcja pojazdów – aplikacja powinna potrafić rozpoznać pojazd w czasie rzeczywistym.
- Śledzenie pojazdów – każdy pojazd powinien być śledzony po rozpoznaniu w celu analizy kierunku jazdy.
- Zliczanie pojazdów – aplikacja w przypadku pokonania przez pojazd określonego miejsca (linii zliczania) powinna zarejestrować przejazd w określonym kierunku. Liczba samochodów, które przekroczyły linię powinna być aktualizowana w czasie rzeczywistym.

c) Interfejs użytkownika

- Komunikaty i informacje zwrotne – użytkownik powinien być w czasie rzeczywistym informowany o stanie aplikacji. W tym celu zostaną stworzone stosowne komunikaty, które będą wyświetlane użytkownikowi na różnych etapach działania aplikacji.

### 2.3.2 WYMAGANIA NIEFUNKCJONALNE

Podobnie jak w przypadku poprzednich wymagań, wymagania niefunkcjonalne również zostaną przedstawione w podziale na odpowiednie kategorie.

#### a) Wydajność

- Czas przetwarzania – aplikacja musi zapewniać przetwarzanie klatek wideo bez żadnego opóźnienia, aby mogła być kompatybilna z przetwarzaniem strumienia wideo na żywo. W przypadku takiej możliwości, wskazane jest również zapewnienie przetwarzania klatek w czasie szybszym niż rzeczywisty, w celu jak najszybszej analizy istniejącego już pliku wideo.
- Obsługa dużych plików – system musi umożliwiać przetwarzanie dużych plików bez zauważalnego spadku płynności działania.

#### b) Skalowalność

- Modularność systemu – architektura aplikacji musi być modularna, umożliwiając łatwe dodawanie nowych funkcji, takich jak dodatkowe rodzaje pojazdów bądź inne zasady analizy wideo.

#### c) Dostępność

- Łatwość obsługi – aplikacja powinna być przystępna w obsłudze dla każdego użytkownika. Interfejs powinien być intuicyjny i łatwy w nauce, umożliwiając szybkie rozpoczęcie pracy przez nowych użytkowników. tym celu zakłada się wykorzystanie biblioteki Tkinter.
- Kompatybilność sprzętowa – system powinien być dostępny dla użytkowników niezależnie od platformy sprzętowej. Powinien zapewniać wsparcie sprzętowe dla urządzeń posiadających kartę graficzną NVIDIA oraz dla tych bez GPU.

#### d) Stabilność

- Niezawodność – aplikacja powinna działać stabilnie nawet przy długotrwałym przetwarzaniu plików wideo. Powinna również być odporna na błędy, a w razie ich wystąpienia musi zwrócić użytkownikowi odpowiedni komunikat błędu, w celu jego wyeliminowania.

## 2.4 WYKORZYSTANE TECHNOLOGIE I BIBLIOTEKI

Wybór odpowiednich technologii i narzędzi programistycznych jest kluczowy do stworzenia skutecznej, dokładnej oraz wydajnej aplikacji do analizy ruchu drogowego. W implementowanej aplikacji postanowiłem skorzystać ze sprawdzonych narzędzi, które zapewniają wysoką wydajność oraz łatwość implementacji. Poniżej zostaną przedstawione technologie i biblioteki, które znalazły zastosowanie w opracowanym projekcie.

### 2.4.1 JĘZYK PYTHON

Jako główny język programowania został wybrany trzeci najpopularniejszy język programowania w Polsce (według Raportu z Badania Społeczności IT 2024 [11]), czyli język Python. Drugą możliwością, którą rozważałem było zastosowanie języka Java. Powodem decyzji o zastosowaniu języka Python była jego wszechstronność oraz względna prostota, w porównaniu do innych języków programowania. Język ten jest wręcz idealnie przystosowany do aplikacji opierających się na przetwarzaniu danych oraz korzystających z algorytmów sztucznej inteligencji. Posiada również szeroki wybór bibliotek, które są kluczowe do działania mojego programu. W momencie podejmowania decyzji dotyczącej wyboru technologii miałem również większą znajomość języka Python niż języka Java, dlatego ostatecznie system został stworzony w tej technologii.

Podczas projektowania systemu zastanawiałem się nad wyborem jednej z kilku najpopularniejszych platform programistycznych (tzw. frameworków), takich jak między innymi Flask oraz Django. Zdecydowałem się jednak na pracę bez korzystania z ogólnodostępnych struktur, gdyż zależało mi na prostocie aplikacji. Również nie było mi potrzebne tworzenie aplikacji webowej, a jednak większość tych frameworków została zaprojektowana w celu uproszczenia tworzenia takich aplikacji.

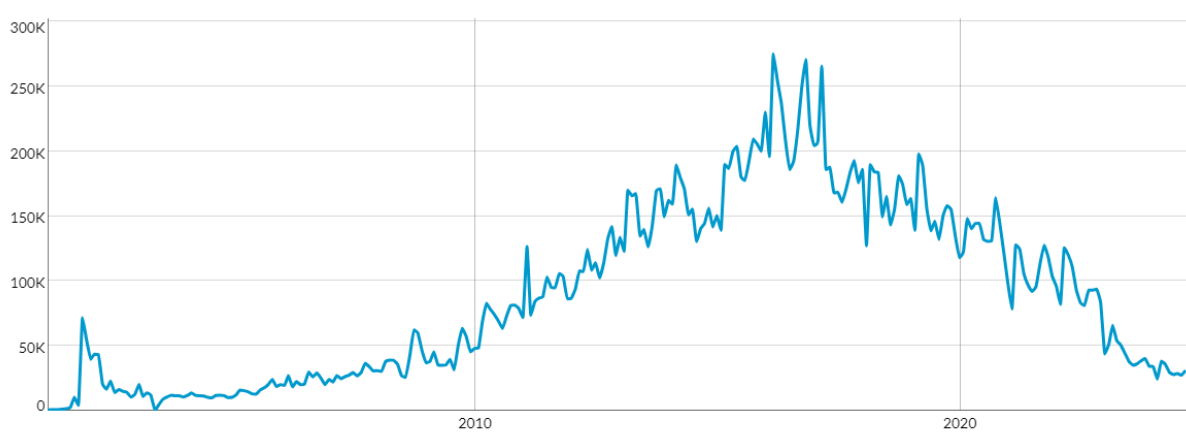
W kontekście tej aplikacji Python jest odpowiedzialny za zarządzanie całą logiką biznesową, interakcję z użytkownikiem oraz interakcję z algorytmami detekcji obiektów. Język ten zarządza wszystkimi elementami aplikacji – rozpoczynając od momentu uruchomienia, przez wybór parametrów na etapie analizy, kończąc na generowaniu raportów. Dzięki swojej

wszechstronności umożliwił stworzenie programu, który niskim kosztem może zostać rozbudowany o kolejne funkcje, w razie zaistnienia takiej potrzeby [12].

## 2.4.2 BIBLIOTEKA OPENCV

OpenCV (Open Source Computer Vision) jest bezpłatną biblioteką do użytku komercyjnego i edukacyjnego. Została stworzona w 1999 roku przez firmę Intel. Jest to jedna z najbardziej popularnych bibliotek stworzonych do przetwarzania obrazu. Według oficjalnej strony producenta, do momentu pisania tej pracy liczba użytkowników przekracza 18 milionów, a sama społeczność uczestnicząca w rozwijaniu biblioteki liczy 47 tysięcy [13]. Sama biblioteka na przestrzeni ostatnich 10 lat zdobyła bardzo dużą popularność, co ukazują statystyki pobrań na stronie SourceForge.net, czyli jednej z najpopularniejszych stron internetowych udostępniających biblioteki (Rys.5).

Biblioteka ta na początku była tworzona z myślą o aplikacjach wizyjnych dla sprzętu komputerowego Intel. Z czasem jednak przekształcona została w projekt open-source (z ang. otwarte oprogramowanie) i zyskała bardzo dużą popularność na świecie. Aktualnie jest wykorzystywana w wielu firmach, takich jak Google czy Microsoft. Znajduje zastosowanie w takich dziedzinach jak robotyka, systemy monitoringu czy systemy rozpoznawania twarzy.



Rysunek 5 - Liczba pobrań biblioteki OpenCV ze strony SourceForge.net na przestrzeni ostatnich 23 lat [14]

W opisywanym systemie biblioteka znalazła kilka zastosowań. Przede wszystkim jest wykorzystywana do odczytu i przetwarzania klatek wideo. Wykorzystując fakt, że film jest



zbiorem wielu następujących po sobie zdjęć, możliwe jest odtworzenie w pętli każdego zdjęcia (bądź też co któregoś zdjęcia, o czym będzie mowa w rozdziale poświęconym optymalizacji) oraz dokonanie na nim działań. Biblioteka OpenCV wspiera najpopularniejsze formaty wideo, takie jak przykładowo MP4 i AVI, a także pozwala na przekształcenie obrazu w celu dostosowania przykładowo jego rozdzielczości. Jest ona także przystosowana do współpracy z popularnymi modelami sztucznej inteligencji [15].

### **2.4.3 MODEL YOLO**

YOLO (You Only Look Once) jest jednym z najbardziej zaawansowanych modeli detekcji obiektów. Został on zaprojektowany na Uniwersytecie Waszyngtońskim przez Josepha Redmona oraz Aliego Farhadiego. Pierwsze wersje modelu zostały wprowadzone w 2015, zyskując od razu popularność dzięki swojej dokładności i szybkości przetwarzania danych. Od tego czasu był on sukcesywnie rozwijany – w momencie pisania tej pracy została zaprezentowana już 11 wersja tego modelu. W moim systemie zaimplementowany został model YOLOv8, wypuszczony na rynek w 2023 roku.

Największymi zaletami tego modelu, które przekonały mnie do jego wykorzystania, są jego szybkość działania oraz dokładność detekcji. Twórcy udostępnili różne jego wersje, które mogą zostać łatwo zmienione, w zależności od potrzeb. W opisywanej aplikacji zastosowałem model w wersji „small”, która charakteryzuje się szybszym działaniem kosztem dokładności. Do wyboru mamy również inne wersje, począwszy od „nano”, która charakteryzuje się najszybszymi obliczeniami, jednak jest stosunkowo mało dokładna aż do „extra large”, która jest najwolniejszym, najbardziej precyzyjnym modelem. Podczas testów aplikacji nie odczułem potrzeby zmiany modelu na bardziej precyzyjny, aczkolwiek taka zmiana może być konieczna w przypadku obróbki materiałów gorszej jakości.

### **2.4.4 BIBLIOTEKA TORCH**

Torch, znana także jako PyTorch jest otwartoźródłową biblioteką służącą do obliczeń numerycznych, która została zaprojektowana z myślą o wsparciu uczenia maszynowego oraz

przetwarzania danych na dużą skalę. PyTorch powstał w 2016 roku jako odpowiedź Facebook AI Research na istniejący już framework TensorFlow.

Jednym z kluczowych atutów tej biblioteki jest jej wsparcie obliczeń dla danych korzystając z procesora karty graficznej komputera (GPU) zamiast procesora komputera, co pozwala na kilkukrotne przyspieszenie procesów obliczeniowych. Dopiero implementacja tej biblioteki i odciążenie procesora w obliczeniach pozwoliło mojemu systemowi działać w czasie rzeczywistym, umożliwiając analizę nie tylko istniejących już nagrań, lecz także korzystając ze strumienia wideo wysyłanego przez kamerę.

#### **2.4.5 BIBLIOTEKA TKINTER**

Podczas projektowania aplikacji bardzo ważne jest zapewnienie końcowemu użytkownikowi interfejsu, który będzie intuicyjny i łatwy do nauczenia. W tym celu wykorzystałem domyślną bibliotekę języka Python do tworzenia graficznych interfejsów użytkownika (GUI), czyli bibliotekę Tkinter. Pozwala ona na tworzenie funkcjonalnych i interaktywnych interfejsów, dzięki czemu użytkownik nie będzie zmuszony do korzystania z linii komend. Tkinter pozwala na użycie gotowych komponentów, takich jak przyciski, pola tekstowe czy okna dialogowe, które mogą zostać łatwo zintegrowane z programem.

Największymi zaletami tej biblioteki, które przekonały mnie do jej użycia w systemie są łatwość konfiguracji oraz natywna integracja z językiem programowania. Jako, iż szybkość działania programu była kluczowa przy jego projektowaniu, nie chciałem dopuścić do sytuacji, w której duża ilość zasobów komputera jest poświęcana na menu konfiguracyjne. Te w założeniu miało być jak najprostsze, aby można było przydzielić więcej zasobów na sam proces analizy wideo.

#### **2.4.6 BIBLIOTEKA PANDAS**

Ostatnią biblioteką, którą wykorzystuję w moim systemie jest biblioteka Pandas. Została ona stworzona do zarządzania, przetwarzania oraz analizowania danych. Została ona wybrana przeze mnie głównie ze względu na łatwe zarządzanie danymi, które są przechowywane w formie tabelarycznej. Użytkownicy podczas analizy wideo tworzą tak zwane DataFrames, które

po jej zakończeniu są zapisywane do arkusza kalkulacyjnego. Ta funkcjonalność jest kluczowa w kontekście całego systemu, gdyż system stworzony do analizy ruchu drogowego bez możliwości zapisywania danych byłby niefunkcjonalny.

### 3. STRUKTURA I FUNKCJONALNOŚĆ APLIKACJI

W tym rozdziale opisana zostanie budowa aplikacji pod względem technicznym, w kontekście jej struktury oraz wykorzystanych rozwiązań. Jego celem jest przedstawienie informacji niezbędnych do zrozumienia struktury aplikacji, wzajemnych zależności poszczególnych warstw i modułów oraz przepływu sterowania i danych. Szczegółowa specyfikacja wewnętrzna znajduje się formie komentarzy w kodzie w plikach źródłowych.

#### 3.1 ARCHITEKTURA APLIKACJI

Architektura aplikacji została zaprojektowana z uwzględnieniem modularności w celu zapewnienia łatwości modyfikacji i rozwoju. W jej skład wchodzi takie komponenty jak:

1. Interfejs użytkownika (zaimplementowany w pliku: `gui.py`): Warstwa, która jest odpowiedzialna za interakcję z użytkownikiem. Została zbudowana w całości na bazie biblioteki Tkinter i umożliwia użytkownikowi konfigurację parametrów analizy oraz wyświetlanie komunikatów i wyników.
2. Moduł analizy i detekcji obiektów (zaimplementowany w pliku: `analysis.py`): Warstwa, która jest odpowiedzialna za odczytywanie i analizę filmu klatka po klatce, a następnie - wykorzystując model YOLO – identyfikację oraz klasyfikację wykrytych obiektów.
3. Moduł śledzenia obiektów (zaimplementowany w pliku: `tracker.py`): Bardzo ważny moduł, odpowiedzialny za śledzenie wykrytych obiektów w kolejnych klatkach. Przeciwdziała duplikatom i zapewnia ciągłość identyfikacji poszczególnych obiektów.
4. Moduł zarządzania modelem (zaimplementowany w pliku: `model_handler.py`): Zajmuje się wstępnym załadowaniem oraz inicjalizacją modelu YOLO.
5. Moduł pomocniczy (zaimplementowany w pliku: `utils.py`): Zawiera różne pomocnicze funkcje, takie jak między innymi obsługę błędów. Po zakończonej analizie zajmuje się zapisem zgromadzonych danych do pliku.

Poniżej została przedstawiona struktura systemu (Rys. 6) – dla przejrzystości usunięte zostały foldery związane z wirtualnym środowiskiem Python, pozostawiając sam rdzeń programu:

```
app/  
├── main.py  
├── gui.py  
├── tracker.py  
├── model_handler.py  
├── analysis.py  
├── utils.py  
├── requirements.txt  
├── coco.txt  
└── yolov8s.pt
```

*Rysunek 6 - Struktura realizowanego systemu przedstawiona na bazie zbioru plików*

### 3.2 INTERFEJS UŻYTKOWNIKA (GUI.PY)

Interfejs użytkownika jest pierwszym punktem kontaktu użytkownika z aplikacją po jej uruchomieniu. Oparty na standardowej bibliotece GUI dla języka Python, czyli bibliotece Tkinter umożliwia użytkownikowi konfigurację parametrów oraz monitorowanie postępów analizy. Moduł ten tworzy interfejs graficzny aplikacji, rozmieszczając przyciski, pola wyboru oraz inne elementy w oknie. Klasa ta posiada również funkcje odpowiadające za pobranie od użytkownika parametrów konfiguracyjnych potrzebnych do przeprowadzenia analizy, takich jak wybór trybu pracy, ustawienie prędkości przetwarzania czy wybór źródła sygnału wideo. Opis poszczególnych elementów interfejsu użytkownika znajduje się w rozdziale 4, poświęconym działaniu systemu.

### 3.3 MODUŁ ANALIZY I DETEKCJI OBIEKTÓW (ANALYSIS.PY)

W tym podrozdziale opisane zostanie działanie rdzenia aplikacji, który odpowiada za przetwarzanie strumienia wideo. Do jego zadań należą:

- a) odczytywanie klatek wideo,
- b) detekcja obiektów za pomocą modelu YOLO, oraz
- c) przekazywanie wyników do modułu śledzenia obiektów.

Moduł został zaprojektowany z myślą o efektywności oraz skalowalności. Korzystając z klasyfikatora YOLO oraz biblioteki PyTorch, jest w stanie przeprowadzić analizę wideo z dużą dokładnością oraz w czasie rzeczywistym w przypadku analizy strumienia wideo a nawet i szybciej w przypadku analizy nagrania. Zaimplementowano tu również mechanizmy redukcji złożoności obliczeniowej, takie jak możliwość pomijania klatek wideo w przypadku, gdy użytkownik wybierze szybszy tryb analizy. Moduł jest przystosowany do działania w trybie wyświetlania oraz w trybie zapisu danych. Szczegółowy opis algorytmu analizy wideo jest zawarty w punkcie 4.2 tej pracy.

### **3.4 MODUŁ ŚLEDZENIA OBIEKTÓW (TRACKER.PY)**

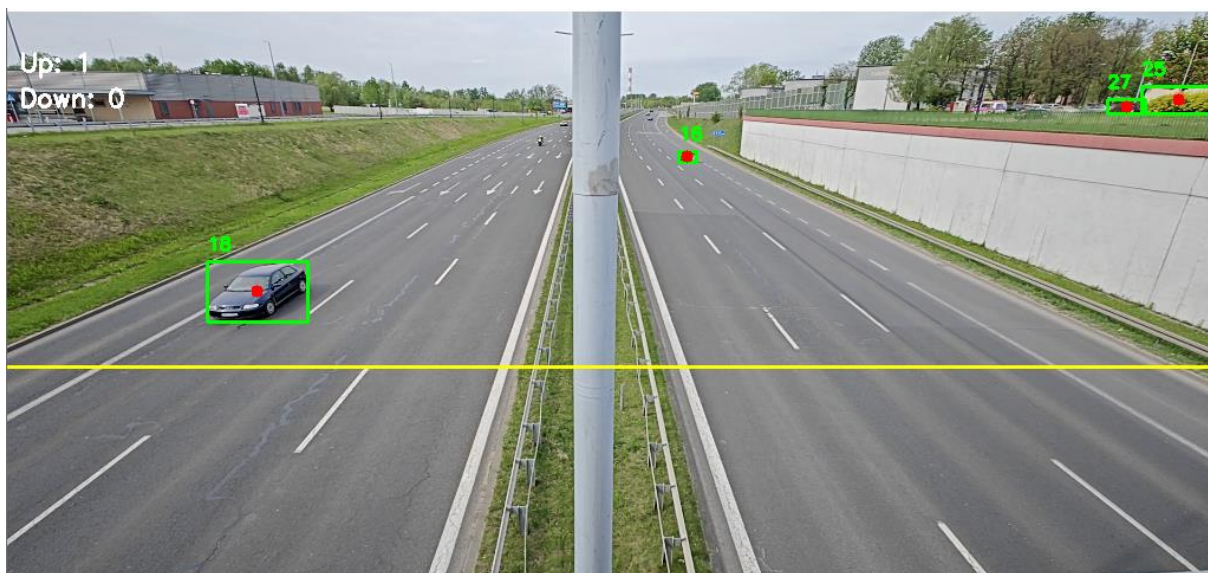
Moduł śledzenia obiektów odpowiada za ciągłość identyfikacji pojazdów w trakcie analizy wideo. Dzięki przypisaniu unikalnych identyfikatorów do wykrytych obiektów oraz śledzenia ich ruchu można zmniejszyć liczbę duplikatów wykryć niemal do zera. Pozwala to również na identyfikację, w jakim kierunku porusza się pojazd.

Podczas inicjalizacji, moduł śledzenia przypisuje wartości progów dopasowania. Nie są one dostępne do konfiguracji przez użytkownika i zostały wstępnie określone arbitralnie – planowane jest poruszenie tego zagadnienia bardziej szczegółowo w ramach pracy magisterskiej. Określają one, jak blisko siebie muszą znajdować się wykrycia obiektów na kolejnych klatkach, aby zostały przypisane do istniejących już obiektów. Następnie inicjuje struktury danych niezbędne do przechowywania informacji o śledzonych obiektach. Po inicjalizacji, moduł ten otrzymuje dane o wykrytych obiektach z modułu analizy i detekcji obiektów. Wśród tych danych znajdują się informacje opisujące między innymi współrzędne tzw. brył brzegowych (ang. bounding box), czyli czegoś w rodzaju ramek otaczających dany obiekt, klasa wykrytego obiektu oraz wskaźnik pewności detekcji.

Po otrzymaniu tych informacji następuje obliczanie współrzędnych środka ciężkości dla każdego obiektu. Po obliczeniu tych współrzędnych, następuje sprawdzenie przemieszczenia się obiektu względem jego położenia na poprzedniej klatce. Moduł wykorzystuje w tym celu wzór na odległość euklidesową oraz jego implementację dostępną w bibliotece „Math” języka Python [16]. Jeśli odległość jest mniejsza niż próg dopasowania, środek ciężkości istniejącego obiektu jest aktualizowany do nowej pozycji. W przypadku przekroczenia tej odległości, system zakłada, że pojawił się nowy obiekt. Następuje wtedy przypisanie nowego, unikalnego

identyfikatora dla tego obiektu. Po przetworzeniu wszystkich wykryć, moduł usuwa identyfikatory, które nie zostały wykryte w obecnej klatce. Pozwala to na zwolnienie części zasobów komputera.

Przykład wykrycia bounding box-ów dla pojazdów (jasno zielone obramowanie w formie prostokąta) oraz wyznaczonych środków ciężkości pojazdu (czerwony punkt wewnątrz ramek zawierających wykryty obiekt) został przedstawiony na rysunku 7. Możemy zaobserwować na nim również identyfikatory przypisane do pojazdów przez moduł śledzenia (w postaci liczb umieszczonych nad ramkami otaczającymi wykryty obiekt). Dodatkowo na rysunku 7 w prawym górnym rogu widoczny jest również przykład błędnego wykrycia obiektu przez klasyfikator polegający na oznaczeniu jako pojazd innych obiektów na obrazie. Tego typu błędy często występują w podobnych projektach – tutaj krzak oznaczony identyfikatorem 25 dla modelu YOLO jest zbliżony wyglądem do samochodu osobowego, przez co nadano mu wskaźnik pewności detekcji na tyle wysoki, aby został przetworzony przez system jako samochód. Jednak takie błędne wykrycia nie mają wpływu na wyniki analizy, co dokładnie zostanie opisane w podrozdziale 4.3 niniejszej pracy. Bardziej szczegółowe zbadanie tego zjawiska planowane jest w ramach realizacji pracy magisterskiej.



*Rysunek 7 - Przykład klatki wideo z włączonym pokazywaniem bounding box-ów oraz ich środków ciężkości*

### 3.5 INNE MODUŁY SYSTEMU (MODEL\_HANDLER.PY I UTILS.PY)

Oprócz wcześniej wspomnianych modułów, system składa się jeszcze z kilku innych, które mimo swojego małego skomplikowania są kluczowe dla działania systemu. W niniejszej sekcji omówione zostaną takie moduły jak Model Handler oraz Utils, które odpowiadają za wsparcie dla głównych funkcjonalności systemu umożliwiając jego efektywne działanie.

Pierwszym modułem, który zostanie opisany jest Model Handler. Odpowiada on za zarządzanie klasyfikatorem detekcji obiektów, którym w przypadku tego systemu jest model YOLO. Głównym jego zadaniem jest inicjalizacja oraz konfiguracja modelu. W zależności od otrzymanych danych, moduł ten wczytuje klasyfikator na procesor lub na kartę graficzną. W przypadku wystąpienia jakichkolwiek błędów, zwraca kod błędu. Dzięki zastosowaniu osobnego modułu do tych zadań, możliwe jest oddzielenie części konfiguracyjnej od modułu analizy. Potencjalnie ułatwi to również aktualizację systemu w przypadku zmiany modelu klasyfikatora na nowszy bądź inny, bez ingerencji w logikę analizy.

Drugim modułem jest moduł funkcji pomocniczych, czyli Utils. Jest to moduł, który pozwala na oddzielenie funkcji niezwiązanych z logiką aplikacji od modułów zajmujących się analizą danych. W tym module znajduje się funkcja odpowiadająca za wyświetlanie komunikatów. Dzięki temu pozostałe moduły mogą przesłać jedynie treść błędu, a ten moduł zajmie się całą logiką wyświetlenia błędu. Zaimplementowana również została funkcjonalność odczytu tekstu z plików tekstowych. W tym systemie jest używana do odczytania listy etykiet z pliku tekstowego, które są wykorzystywane przez model YOLO do klasyfikacji wykrytych obiektów. Ostatnią funkcjonalnością tego modułu jest zapis zgromadzonych danych do arkusza kalkulacyjnego. Moduł ten otrzymuje z modułu analizy dane, które następnie konwertuje na obiekt DataFrame biblioteki „Pandas”. Po udanej konwersji zapisuje obiekt do pliku, który został wskazany przez użytkownika na początku analizy.



## **4 DZIAŁANIE SYSTEMU I ANALIZA ROZWIĄZAŃ**

W tym rozdziale skupię się na działaniu aplikacji z perspektywy użytkownika korzystającego z niej. Skupię się również na działaniu linii zliczania i postaram się wyjaśnić, jak działa algorytm detekcji pojazdów.

### **4.1 GRAFICZNY INTERFEJS UŻYTKOWNIKA**

Kluczem do zapewnienia uniwersalności programu było udostępnienie użytkownikowi wielu opcji i umożliwienie konfiguracji procesu analizy tak, by program znalazł zastosowanie w wielu scenariuszach i był w stanie przetwarzać różnorodne przypadki i rodzaje filmów. Wygląd interfejsu użytkownika został przedstawiony na rysunku 8, a poniżej opisane zostaną wszystkie opcje konfiguracyjne, które zostały uwzględnione w interfejsie:

- Wybór urządzenia do wykonania obliczeń: Użytkownik ma możliwość wyboru czy obliczenia będą wykorzystywały zasoby procesora urządzenia czy karty graficznej. W większości przypadków wybór karty graficznej jako trzonu obliczeń jest opcją wydajniejszą, aczkolwiek w niektórych scenariuszach użytkownik może chcieć skorzystać z opcji wykorzystania procesora urządzenia, dlatego też pozostawiono wybór pomiędzy tymi podzespołami. Poniżej tej opcji wyboru wyświetlana jest informacja, które urządzenie zostało wybrane.
- Wybór trybu pracy: System oferuje pracę w dwóch trybach. Tryb „Display” wyświetla okno analizy bez zapisu danych. Jest to tryb przewidziany głównie dla sprawdzenia poprawności konfiguracji. Użytkownik może zobaczyć, czy przykładowo optymalnie ustalił pozycję linii zliczającej, zanim przejdzie do pełnoprawnej analizy. Tryb „Record” natomiast przewidziany jest do pełnoprawnej analizy. Po naciśnięciu przycisku wywołane zostaje okno dialogowe, w którym użytkownik wybiera nazwę oraz lokalizację pliku, w którym będą zapisane wyniki analizy. Po uruchomieniu analizy użytkownik nie ma podglądu na jej przebieg. Pozwala to na przeprowadzenie badań szybciej, gdyż zasoby komputera, które zajmowałyby się wyświetlaniem obrazu zostają uwolnione. Po zakończonej analizie zebrane dane zostają zapisane we wcześniej wybranym pliku, a użytkownik zostaje poinformowany o zakończonym procesie.

- Orientacja linii zliczania: W celu zachowania jak największej uniwersalności aplikacji użytkownik może wybrać, czy linia zliczania ma zostać ustawiona pionowo, czy poziomo. W przypadku poziomej linii, aplikacja zlicza pojazdy jadące „w górę” i „w dół”, a w przypadku pionowej „w prawo” i „w lewo”.
- Miejsce położenia linii zliczania w kadrze: Użytkownik może wybrać, gdzie dokładnie będzie znajdować się linia, po przekroczeniu której następuje zliczanie pojazdów. Ważną informacją odnośnie wyboru miejsca położenia linii jest fakt, że klatka wideo zostaje przeskalowana do wymiarów 1024 na 480 pikseli. Liczenie pikseli rozpoczyna się w lewym górnym rogu ekranu, więc w przypadku linii pionowej skrajnie lewą wartością będzie 0, a skrajnie prawą 1023. Analogicznie w przypadku linii poziomej – wartość 0 oznacza umiejscowienie linii jako górnej ramki ekranu, a 479 jako dolnej ramki. Mechanizm działania linii zliczania zostanie opisany w dalszej części pracy.
- Wybór typów pojazdów: Aplikacja przewiduje możliwość analizowania materiału wideo pod kątem poruszania się wybranych typów pojazdów. Na potrzeby tej pracy wybór został ograniczony do 3 rodzajów pojazdów, aczkolwiek nic nie stoi na przeszkodzie, aby umożliwić użytkownikowi szerszy wybór.
- Wybór prędkości przetwarzania: Początkowo ta opcja została uwzględniona w celu przyspieszenia przetwarzania filmu. Jednak po implementacji PyTorch i przeniesieniu obliczeń na kartę graficzną nie była rozwinięta do takiego poziomu, aby zwracać bardzo dokładne wyniki, dlatego pozostawiłem ją jedynie w formie testowej. Jedyne scenariusz, w którym jest bardzo przydatna to obróbka filmu nagranych w 60 klatkach na sekundę – wtedy wyrównujemy prędkość przetwarzania do filmu nagranych w 30 klatkach na sekundę, nie tracąc dokładności. Dokładniejsze zbadanie działania tej opcji planowane jest w ramach pracy magisterskiej.
- Wybór źródła wideo: Założeniem funkcjonalnym aplikacji było umożliwienie analizy zarówno nagranych wcześniej filmów, jak i materiału w postaci strumienia wideo na żywo. Udało się te założenie zrealizować. Użytkownik może wybrać film nagrany w wielu najpopularniejszych formatach, bądź też obraz na żywo pochodzący np. z kamery internetowej.
- Ustawienie interwału czasowego: Domyślnie jest to jedna minuta, czyli aplikacja będzie co 60 sekund resetować licznik i – w przypadku trybu „Record” – zapisywać zgromadzone dane do struktury DataFrame. Użytkownik może edytować ten czas według potrzeb.

- Przycisk „Uruchom”: Inicjuje proces analizy, uruchamiając wątek przetwarzający wideo.
- Przycisk „Zamknij”: Zamyka aplikację.
- Konsola wyjściowa: Wyświetlane są na niej najważniejsze komunikaty, takie jak błędy, informacje o wybranych opcjach konfiguracyjnych oraz postęp procesu analizy.

## 4.2 ALGORYTM DETEKCJI OBIEKTÓW

W tym podrozdziale opisane zostanie działanie rdzenia aplikacji, który odpowiada za przetwarzanie strumienia wideo. Do jego zadań należą:

- a) odczytywanie klatek wideo,
- b) detekcja obiektów za pomocą modelu YOLO, oraz
- c) przekazywanie wyników do modułu śledzenia obiektów.

Na rysunku 9 został przedstawiony diagram czynności algorytmu działania modułu analizy i detekcji obrazów wraz z opisem poszczególnych kroków.

Konfiguracja analizy

Wybierz urządzenie:

☒ CPU  
☐ GPU

Zastosuj

Używane urządzenie: CPU

Wybierz tryb pracy:

Display Record

Wybierz orientację linii zliczającej:

☒ Pozioma  
☐ Pionowa

Podaj miejsce linii zliczającej:

Wybierz typy pojazdów do zliczania:

☐ Samochody  
☐ Ciężarówki  
☐ Motocykle

Wybierz prędkość przetwarzania:

☒ Standardowa  
☐ Szybka (pomijaj co drugą klatkę)

Wybierz źródło wideo:

Wybierz plik wideo ☐ Użyj kamery

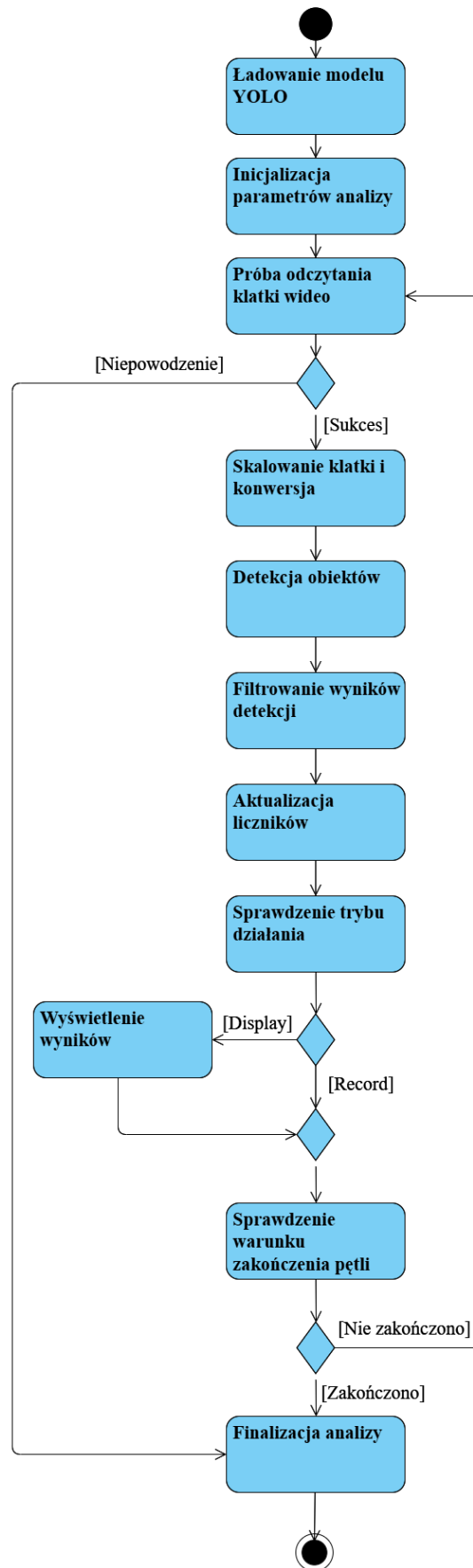
Dodatkowe opcje:

Ustaw interwał czasowy (min)

Uruchom

Zamknij

Rysunek 8 - Graficzny interfejs użytkownika zrealizowanej aplikacji



Rysunek 9 - Diagram aktywności ilustrujący działanie algorytmu detekcji obiektów

Opis poszczególnych kroków algorytmu:

1. Ładowanie modelu YOLO: Pierwszym krokiem opisywanego algorytmu jest jego inicjalizacja, w czasie której następuje załadowanie wcześniej „wytrenowanego” klasyfikatora YOLO, który w tym systemie jest odpowiedzialny za detekcję obiektów. W zależności od dostępnych zasobów systemowych oraz ustawień model jest przygotowywany do pracy korzystając z procesora bądź karty graficznej. Cały proces inicjalizacji jest nadzorowany w celu zapewnienia poprawności uruchomienia analizy oraz poinformowania użytkownika w przypadku wystąpienia błędów.
2. Inicjalizacja parametrów analizy: Po pomyślnym załadowaniu klasyfikatora aplikacja przechodzi do inicjalizacji źródła wideo, na którym będzie dokonywana analiza. Do tego celu jest wykorzystywana biblioteka OpenCV, która zapewnia gotowe klasy służące do przechwytywania wideo [17]. Proces ten obejmuje sprawdzenie poprawności ścieżki do pliku lub w przypadku wyboru strumienia na żywo sprawdzenie czy kamera internetowa jest poprawnie podłączona do komputera. Dodatkowo moduł odczytuje takie informacje, jak między innymi liczba klatek na sekundę oraz całkowita liczba klatek wideo. Te informacje są kluczowe dla systemu, aby móc zapewnić poprawną synchronizację analizowanych danych. Ten krok algorytmu również przygotowuje system do odczytu klatek wideo, poprzez ustawienie wskaźnika na początek strumienia oraz zarezerwowania przestrzeni w pamięci na przechowywanie zgromadzonych danych. Następuje również inicjalizacja liczników, które będą odpowiedzialne za rejestrowanie liczby przejeżdżających pojazdów. Dzięki temu aplikacja jest w pełni gotowa na rozpoczęcie analizy klatek wideo w pętli.
3. Próba odczytania klatki wideo: W tym miejscu rozpoczyna się główna pętla realizująca analizę. System iteruje przez każdą klatkę strumienia wideo i przeprowadza na niej proces detekcji obiektów. Jeśli odczytanie klatki zakończy się niepowodzeniem (przykładowo w sytuacji, gdy po odczytaniu ostatniej klatki wideo następnej już nie będzie), następuje finalizacja algorytmu – pętla jest przerywana, a w przypadku wybrania odpowiedniego trybu zgromadzone dane są zapisywane w arkuszu kalkulacyjnym.

4. Skalowanie klatki i konwersja: W celu optymalizacji procesu, każda odczytana klatka jest skalowana do odpowiedniego rozmiaru, zgodnego z wymaganiami klasyfikatora. W przypadku tej aplikacji, klatki są skalowane do wymiaru 1024 na 400 pikseli. Pozwala to na zwiększenie prędkości analizy nie wpływając znacznie na dokładność detekcji. Również dzięki temu aplikacja nie musi wymagać od użytkownika materiału nagranych w konkretnej rozdzielczości – będzie działać identycznie w przypadku obrazu nagranych w rozdzielczości zarówno HD, jak i 4K. Przeskalowana klatka jest następnie konwertowana do tak zwanego formatu tensorowego (zbliżonego strukturą do tablic czy macierzy), który jest kompatybilny z modelem YOLO. Cały proces opiera się na normalizacji wartości pikseli oraz zmianie kolejności kanałów kolorów, aby zapewnić jak najlepsze warunki do detekcji obiektów.
5. Detekcja obiektów: Przetworzona klatka jest przekazywana do klasyfikatora w celu detekcji obiektów. Model dokonuje analizy klatki, po której zwraca wyniki detekcji. Wynikiem takiej detekcji są współrzędne tzw. brył brzegowych (ang. bounding box) oraz klasy wykrytych obiektów. W wynikach zawarta jest również dokładność detekcji, która może być wykorzystywana do odrzucenia mniej pewnych wykryć.
6. Filtrowanie wyników detekcji: Wynikiem detekcji realizowanej przez klasyfikator są wszystkie obiekty, które klasyfikator jest w stanie rozpoznać. Przykładowo, analizując klatkę wideo obejmującą obszar drogi klasyfikator zwróci nie tylko obiekt samochodu, ale także drzewo, pieszego lub znak drogowy. Dlatego w celu wydobycia z nich przydatnych w danej aplikacji obiektów, należy przeprowadzić tzw. przetwarzanie wtórne (ang. post-processing). Proces ten obejmuje filtrowanie wykrytych obiektów na podstawie wybranych przez użytkownika klas pojazdów (samochody, ciężarówki oraz motocykle). Dzięki eliminacji wykryć należących do niepotrzebnych klas, system może skoncentrować analizę tylko na tych istotnych z punktu widzenia systemu obiektach. Przefiltrowane obiekty są następnie przekazywane do modułu śledzenia.
7. Aktualizacja liczników: System po otrzymaniu informacji z modułu śledzenia aktualizuje liczniki ruchu. Rejestrują one przejścia pojazdów przez wyznaczoną linię oraz określają ich kierunek ruchu analizując położenie na poprzedniej klatce. W przypadku wybrania trybu „Display” system wyświetla bieżącą klatkę wraz z dodatkowymi informacjami. W przypadku drugiego trybu dane z analizy obecnej klatki zostają zapisane do struktury DataFrame.

8. Sprawdzenie warunku zakończenia pętli: Pętla może zostać zakończona po wciśnięciu odpowiedniego przycisku, który odpowiada za przerwanie wykonywanego procesu analizy. Po jego naciśnięciu analiza jest kończona, a użytkownik zostaje przekierowany do głównego interfejsu aplikacji. W przypadku braku przerwania działania pętli, analiza obecnej klatki zostaje zakończona, a system przechodzi do analizy następnej klatki ze strumienia.

### **4.3 DZIAŁANIE MECHANIZMU OPARTEGO NA LINII ZLICZANIA**

W tym podrozdziale zamieściłem opis działania tak zwanej „linii zliczającej”, czyli mojego pomysłu na optymalizację analizy i poprawę dokładności w opisywanej aplikacji. Sam pomysł utworzenia linii, po przejechaniu której nastąpi zliczenie pojazdu pojawił się po pierwszych niepowodzeniach w implementacji opisywanej aplikacji. Początkowo w realizacji zadania wykorzystałem algorytm śledzenia pojazdów oparty na analizie zmian położenia środków ciężkości obiektów (ang. Centroid Tracking algorithm). Zastosowany wstępnie algorytm jest rozwiązaniem względnie prostym i mało obciążającym komputer, natomiast nie należy do najdokładniejszych algorytmów. Dużo dokładniejszym algorytmem jest na przykład algorytm Deep SORT, jednakże jest on bardziej wymagający.

Podczas testowania pierwszych wersji aplikacji, bez użycia linii zliczającej wyniki były niedokładne. Często występował problem z detekcją i śledzeniem odległych pojazdów, gdyż były to zbyt małe i niewyraźne punkty dla klasyfikatora. Następowало wtedy wielokrotne zliczanie tego samego pojazdu. Problem można było rozwiązać użyciem dokładniejszego klasyfikatora, jednak sprawiało to, że aplikacja działała dużo wolniej i nie spełniała założonych wymagań.

Korzystając z obserwacji, że dokładność detekcji oraz śledzenia wzrasta wraz z spadkiem odległości obiektu do kamery, postanowiłem wykluczyć z analizy te miejsca, w których dokładność detekcji była najmniejsza. Poskutkowało to utworzeniem wycinka, którego współrzędne są ustalane przed rozpoczęciem analizy przez użytkownika. Aktualizacja liczników odbywa się tylko i wyłącznie wtedy, kiedy centroid obiektu znajduje się dokładnie w tych współrzędnych, powiększonych o bufor, czyli o kilka pikseli w dół oraz w górę (lub w prawo i lewo, w przypadku pionowej linii). W przypadku tej aplikacji bufor jest ustawiony na stałe i wynosi 10 pikseli. Zastosowanie mechanizmu linii zliczającej pozwala na zniwelowanie



skutku wcześniejszych wykryć obiektów, gdyż następują one poza zadeklarowanymi współrzędnymi i nie są brane pod uwagę przy aktualizacji liczników.



*Rysunek 10 - przykład klatki wideo podczas analizy*

Przykład linii zliczającej można zaobserwować na rysunku 10. Współrzędne zostały ustalone przez użytkownika na 300 pikseli. Oznacza to, że wykrycie pojazdu spowoduje zmianę liczników tylko i wyłącznie w sytuacji, kiedy jego środek ciężkości znajdzie się na wysokości pomiędzy 290 a 310 pikseli.

W przypadku klatki przedstawionej na tym rysunku, do modułu śledzenia trafiają współrzędne wszystkich pojazdów znajdujących się na tej klatce. Moduł zaktualizuje istniejące identyfikatory oraz doda nowe, a następnie zwróci te informacje do modułu analizy. Tam też nastąpi porównanie środków ciężkości wykrytych obiektów z współrzędnymi linii zliczającej. Pomimo wykrycia wielu obiektów (możliwe również, że dalsze obiekty zostały wykryte kolejny raz jako nowe) liczniki pozostaną bez zmian, ponieważ żaden obiekt nie znajduje się pomiędzy współrzędnymi początku i końca linii. Można natomiast założyć, że najprawdopodobniej kilka klatek później szary samochód znajdujący się po lewej stronie drogi znajdzie się w odpowiednim miejscu i nastąpi zaktualizowanie licznika. Może on znajdować się we współrzędnych linii zliczającej przez kilka klatek, jednak dzięki działaniu modułu śledzenia będzie on traktowany jako jeden i ten sam pojazd, co pozwoli na uniknięcie duplikatów. Sytuacja została zaprezentowana na rysunku numer 11.



*Rysunek 11 - Przykład klatki wideo podczas analizy z widoczną linią zliczania*

#### **4.4 TRYB „RECORD” – ZAPIS DANYCH**

Z założenia aplikacja miała służyć do gromadzenia danych na temat ruchu drogowego, dlatego ważne było utrwalenie danych na przykład w arkuszu kalkulacyjnym. Z czasem zauważyłem, że najlepszym rozwiązaniem byłoby rozdzielenie aplikacji na dwa osobne tryby działania – jeden przeznaczony do testowania działania, a drugi do przeprowadzania analizy. Pozwoliło to na usunięcie z drugiego trybu funkcjonalności związanej z wyświetlaniem obrazu w czasie rzeczywistym, gdyż użytkownik najprawdopodobniej wszystkie ustawienia konfiguracyjne dobierze korzystając z trybu „Display”. Pozwoli to oszczędzić zasoby komputera i przeprowadzić analizę szybciej.

Po wybraniu trybu „Record” w interfejsie użytkownik zostanie zapytany przez system, w jakim miejscu ma zostać zapisany plik oraz jaka będzie jego nazwa. Aplikacja zapamiętuje wprowadzone dane, jednak tworzy plik dopiero po zakończeniu analizy. Pozwala to uniknąć tworzenia pustych arkuszy, gdy użytkownik wybierze inny tryb bądź też zrezygnuje z uruchomienia aplikacji. Po rozpoczęciu analizy system rozpoczyna przetwarzanie strumienia wideo.

Z związku z faktem, że przetwarzanie wideo w trybie „Record” nie wyświetla nagrania użytkownikowi, kluczowym było wyświetlanie komunikatów w konsoli, aby użytkownik miał świadomość, ile materiału zostało już przetworzone. System zlicza, ile łącznie klatek zawiera wybrany materiał a następnie rejestruje, ile klatek zostało przetworzone. Pozwala to zwracać

użytkownikowi informację, ile procent materiału zostało już przetworzone. Po zakończonym przetwarzaniu dane są zapisywane do arkusza kalkulacyjnego, a użytkownik zostaje poinformowany o tym fakcie stosownym komunikatem, widocznym na rysunku 11. Dane zostają zapisane w dwóch kolumnach. W przypadku wybrania orientacji linii jako pozioma, tworzone są kolumny o nazwach „W górę” oraz „W dół”, a w przypadku pionowej linii „W prawo” i „W lewo”. Do tych kolumn co określony przez użytkownika interwał dodawane są liczby zarejestrowanych pojazdów. To, jak wyglądają zapisane dane w arkuszu kalkulacyjnym przedstawia rysunek numer 12.

```

Urządzenie: Wybrano urządzenie: CUDA
Tryb pracy: Ustawiono tryb: display
Tryb pracy: Ustawiono tryb: record
Prędkość przetwarzania: Ustawiono prędkość: Standardowa
Miejsce linii zliczającej: Ustawiono miejsce linii : 300
Status: Model załadowany na CUDA.
Status: Wideo zostało otwarte pomyślnie.
Status: Rozpoczęto analizę...
Status: 10% przetworzonych klatek.
Status: 20% przetworzonych klatek.
Status: 30% przetworzonych klatek.
Status: 40% przetworzonych klatek.
Status: 50% przetworzonych klatek.
Status: 60% przetworzonych klatek.
Status: 70% przetworzonych klatek.
Status: 80% przetworzonych klatek.
Status: 90% przetworzonych klatek.
Status: 100% przetworzonych klatek.
Status: Dane zostały zapisane do C:/Users/Kamil/Documents/Inz/Test.xlsx
Status: Analiza ruchu drogowego została zakończona.

```

Rysunek 12 - Informacje zwracane przez system do konsoli wyjściowej dla przykładowej analizy

| Czas                             | W górę   | W dół |
|----------------------------------|----------|-------|
| 0                                | 15       | 13    |
| 1                                | 20       | 9     |
| 2                                | 12       | 10    |
| 3                                | 23       | 9     |
| 4                                | 25       | 17    |
| Całkowity czas przetwarzania (s) | 228,5515 |       |

Rysunek 13 - Przykładowe dane z arkusza kalkulacyjnego po zakończonej analizie

Kluczową rzeczą przy zapisywaniu danych jest zadbanie o poprawną synchronizację zapisywanych danych. Nie można po prostu zapisywać interwałów czasowych liczonych na podstawie czasu, który upłynął w rzeczywistości. Byłby to błąd, ponieważ aplikacja na każdym

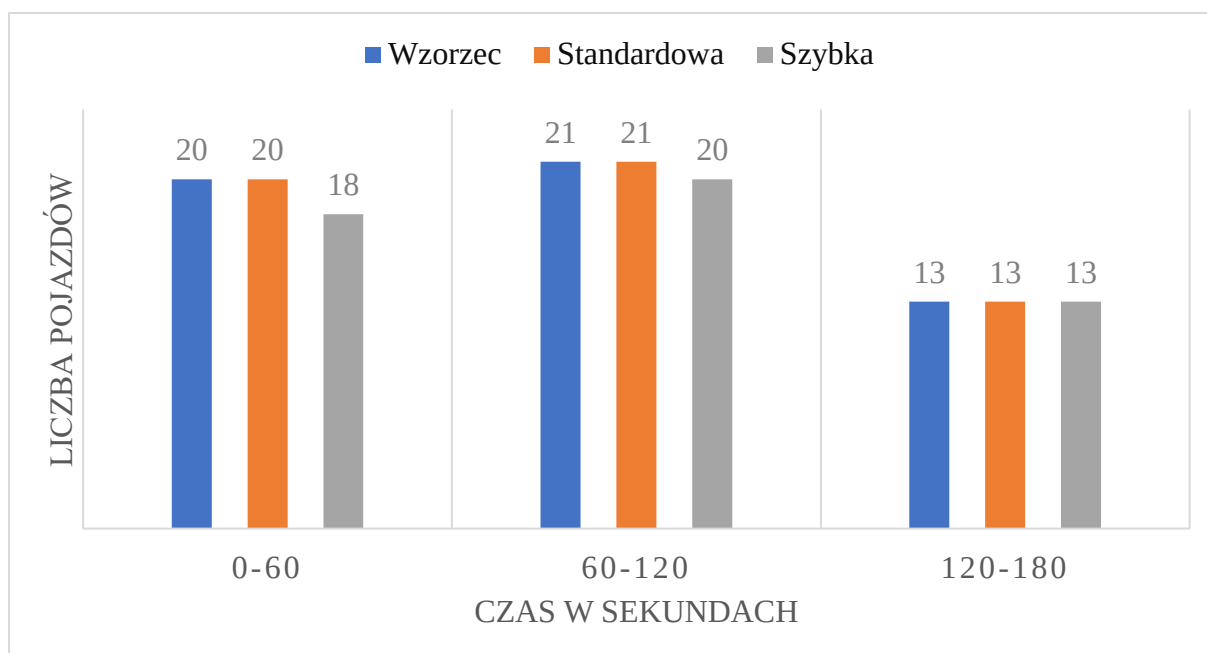
komputerze będzie działać z inną prędkością. Na jednym będzie przetwarzać minutę nagrania w 10 sekund, a na drugim może to zająć 3 minuty. Dlatego aplikacja liczy upływ czasu na nagraniu na podstawie przetworzonych klatek. Po sprawdzeniu, w ilu klatkach na sekundę został nagrany film system oblicza, ile klatek znajduje się w minucie (bądź w innym, wybranym interwale czasowym) nagrania. Pozwala to zachować poprawne odstępy czasowe, niezależnie od wykorzystywanego sprzętu.

## 5. PODSUMOWANIE

Celem niniejszej pracy było zaprojektowanie i wykonanie systemu informatycznego przeznaczonego do przetwarzania oraz analizy natężenia ruchu drogowego na podstawie wcześniej nagranych filmów wideo, bądź strumienia wideo. Dobór technologii został idealnie dopasowany do specyfiki realizowanego projektu, co pozwoliło na sprawną realizację wyznaczonego zadania oraz wszystkich wymagań określonych w pracy.

### 5.1. WNIOSKI

Niniejsza praca opisuje proces projektowania oraz wykonania aplikacji, wraz z dokładnym opisem poszczególnych kroków. Wszystkie wymagania stawiane systemowi realizowanego w ramach pracy zostały zrealizowane, a określony cel został osiągnięty. System spełnia założenia funkcjonalne i нефункционалне, które zostały zdefiniowane na etapie projektowania, co czyni go w pełni użytecznym narzędziem. Podczas procesu projektowania systemu skupiłem się na dostarczeniu funkcjonalności, które pozwolą użytkownikowi na sprawne wykonywanie zadań związanych z tematyką projektu. Do takich zadań należy analiza wideo wraz z detekcją znajdujących się na nim obiektów. Dzięki zastosowaniu modularnej architektury każdy z komponentów systemu może być łatwo modyfikowany i rozwijany. Takie podejście przede wszystkim zwiększa skalowalność aplikacji, ale też znacząco upraszcza jej budowę. Podczas tworzenia systemu szczególną uwagę zwróciłem również na jego wydajność oraz dokładność. System jest w stanie działać w czasie rzeczywistym oraz zwraca bardzo dokładne wyniki analizy, oraz jest w stanie skutecznie śledzić ich ruch, co pozwala uniknąć wielokrotnego zliczania jednego pojazdu. Umożliwia to także zbadanie kierunku ich poruszania się, co również znajduje zastosowanie w moim systemie. Przykładowe wyniki badania natężenia ruchu drogowego wraz z wzorcem zostały przedstawione na rysunku 13.



Rysunek 14 - Statystyki przejeżdżających pojazdów „w górę” po analizie przykładowego filmu

## 5.2 PLAN ROZWOJU SYSTEMU ORAZ MOŻLIWE ZASTOSOWANIA

Dzięki przygotowaniu systemu pod przyszły rozwój możliwe jest łatwe przekształcenie jego działania w zależności od wyznaczonych zadań. W opisywanej formie aplikacja doskonale nadaje się do sporządzania statystyk natężenia ruchu drogowego na podstawie filmu bądź strumienia wideo. Nic jednak nie stoi na przeszkodzie, aby po niewielkich modyfikacjach aplikacji służyła przykładowo do wykrywania wykroczeń na drogach, takich jak jazda pod prąd (na przykład na drogach jednokierunkowych). System mógłby monitorować ruch pojazdów korzystając z kamery, a w przypadku wykrycia pojazdu poruszającego się w przeciwną stronę zwróciłby użytkownikowi dokładny czas wystąpienia tego zdarzenia. Aplikację można rozwijać również w tej samej dziedzinie w jakiej została stworzona, tak aby była jeszcze szybsza i bardziej dokładna. Rozwojem tego konkretnego systemu o te oraz pozostałe zagadnienia planuję zająć się w ramach pracy magisterskiej.

## 6. BIBLIOGRAFIA

1. Piotr Kunikowski, Andrzej Szyszło, Jan Zieliński, Paweł Tutka. „Ruch Drogowy 2020/2021”. Opracowanie wykonane na zlecenie Generalnej Dyrekcji Dróg Krajowych i Autostrad.
2. CMS Cameron McKenna Nabarro Olswang Pośniak i Bejm sp.k. „Raport branży motoryzacyjnej 2024/2025”.
3. Strona internetowa producenta Flir. <https://www.flir.eu/> [dostęp 08.12.2024].
4. Strona internetowa producenta Vivotek. <https://www.vivotek.com/> [dostęp 08.12.2024].
5. Vivotek. Traffic Solutions. [https://vivotek.pl/wp-content/uploads/2024/02/2023\\_traffic\\_solutions\\_en\\_ver.1.pdf](https://vivotek.pl/wp-content/uploads/2024/02/2023_traffic_solutions_en_ver.1.pdf) [dostęp 08.12.2024].
6. Strona internetowa systemu PYRO producenta Eco-Counter. <https://www.eco-counter.com/produits/pyro-evo-range-en/> [dostęp 11.01.2025].
7. Strona internetowa systemu MULTI producenta Eco-Counter. <https://www.eco-counter.com/produits/multi-range/> [dostęp 11.01.2025].
8. Strona internetowa producenta aplikacji CountThings. <https://countthings.com/> [dostęp 20.11.2024].
9. Krzysztof Różanowski. „SZTUCZNA INTELIGENCJA: ROZWÓJ, SZANSE I ZAGROŻENIA”.
10. Definicja słowa klasyfikator. <https://mfiles.pl/pl/index.php/Klasyfikator> [dostęp 19.11.2024].
11. Strona internetowa, na której zawarte są wyniki Raportu z Badania Społeczności IT. <https://bulldogjob.pl/it-report/2024/technologie> [dostęp 21.11.2024].
12. Dokumentacja języka Python. <https://docs.python.org/3/> [dostęp 18.12.2024].
13. Strona internetowa biblioteki OpenCV. <https://opencv.org/about/> [dostęp 16.12.2024]
14. Statystyki pobrań biblioteki OpenCV. <https://sourceforge.net/projects/opencvlibrary/files/stats/timeline?dates=2001-03-01%20to%202024-12-01&period=monthly> [dostęp 01.12.2024]
15. Mateusz Zimoch. „OpenCV. Kurs video. Przetwarzanie obrazów w języku Python”.
16. Dokumentacja funkcji math.hypot. <https://docs.python.org/3/library/math.html#math.hypot> [dostęp 16.12.2024]

17. Dokumentacja

klasy

VideoCapture.

[https://docs.opencv.org/3.4/d8/dfe/classcv\\_1\\_1VideoCapture.html](https://docs.opencv.org/3.4/d8/dfe/classcv_1_1VideoCapture.html) [dostęp 12.12.2024]